



POLITECNICO
MILANO 1863

DIPARTIMENTO DI ELETTRONICA
INFORMAZIONE E BIOINGEGNERIA

Industrial automation and robotics

Sequential Function Chart

24.10.2025 | Paolo Rocco

Contents

1. Elements of the SFC
2. Programming with SFC
3. Examples

Elements of the SFC

01

Sequential Function Chart

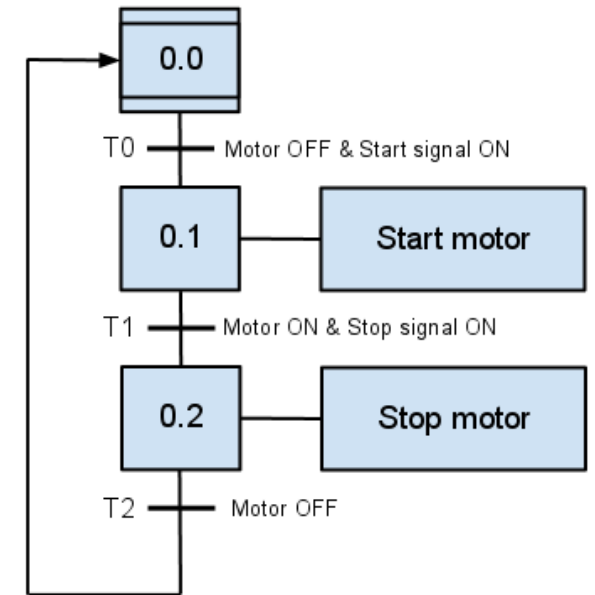
In the 70s of the last century a graphic language was developed for specification of logic control system.

This language was called **Grafcet**

Grafcet gives a formal specification of the evolution and the input/output behavior of a system, from which it is possible to obtain unambiguously the executable code for a synchronous machine interfaced to the outside world (the PLC).

Grafcet is still one of the standard languages for modelling discrete processes and has been included in IEC 1131-3 under the name **Sequential Function Chart**.

Compared to Ladder Diagrams, SFC are not directly usable to program a PLC, rather to specify the expected behavior of a logic system, through simple interconnections of **steps** and **transitions**.



Some of these slides are based on the lectures of Prof. Alberto Leva (Politecnico di Milano)

Sequential Function Chart

The fundamental elements of a SFC are:

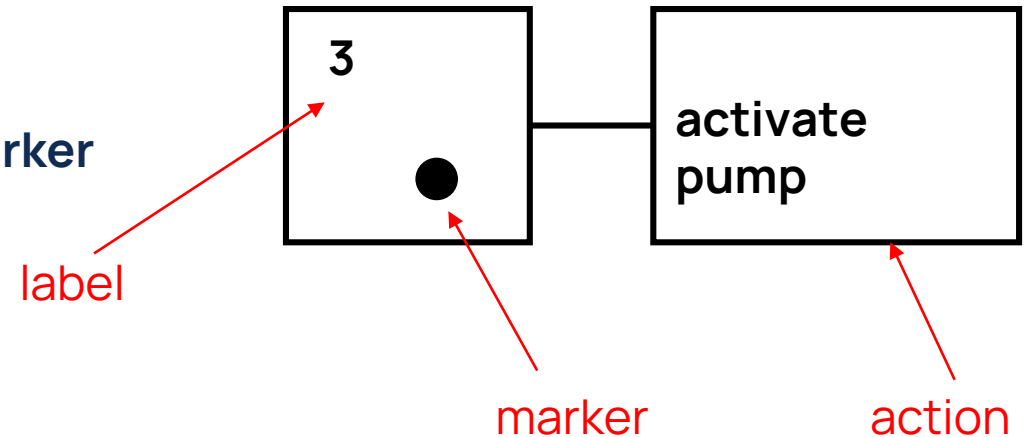
- phases or steps or stages (étapes),
- transitions
- oriented arcs connecting steps to transitions or vice versa,
- rules of evolution, which unambiguously define the dynamic behavior of the program in its capacity as implementation of a DES
- actions associated with the steps
- logical conditions associated with transitions

The state of activation of the steps represents the system state. It is modified by the occurrence of events, which through transitions bring the system into a new condition (with other active steps).

The step

A **step** in a SFC:

- is symbolized by a **square** (with a double border if it is an initial step, i.e. it is active at the beginning of the execution of the program)
- has a **label**, usually consisting of a number
- may be **active** or **inactive**; when active it contains a **marker** (token)
- it can have associated **actions**



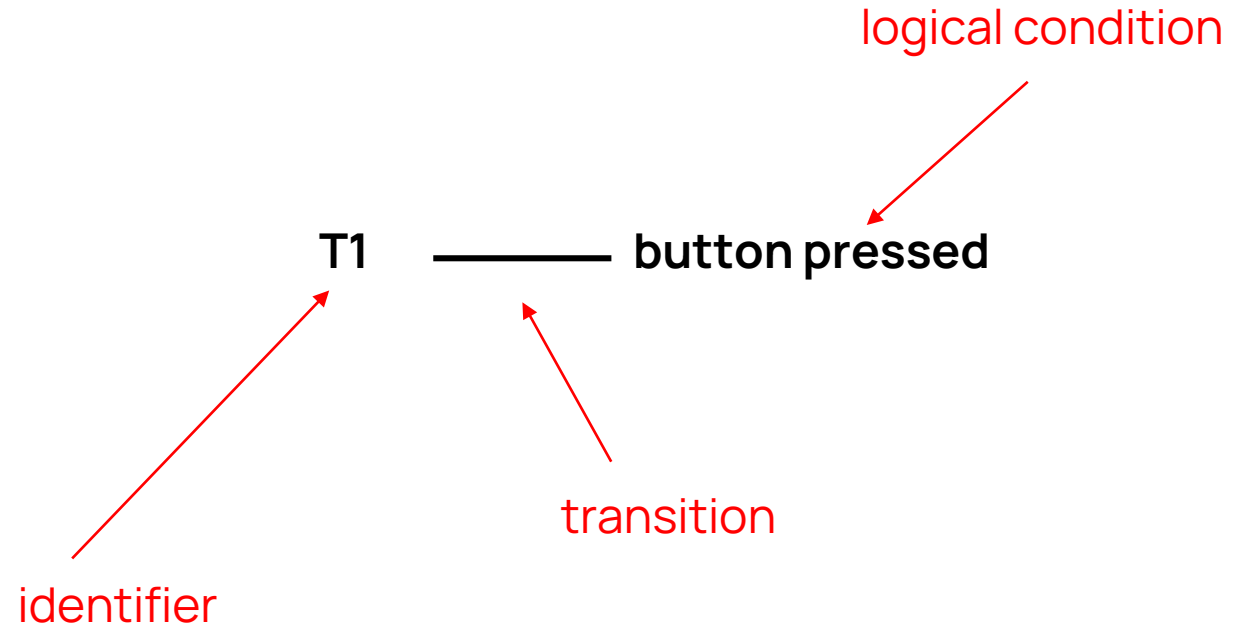
The marker is just a graphical representation of a boolean variable that is associated to the step and contains the information whether the step is active or not: **Step_Name.X**

Also a time variable is associated to each step, measuring the time since last activation of the step: **Step_Name.T**

The transition

A **transition** in a SFC:

- is symbolized by a **horizontal bar**
- has an **identifier**
- can have an **associated logical condition**
- can be **enabled** or not, **superable** or not



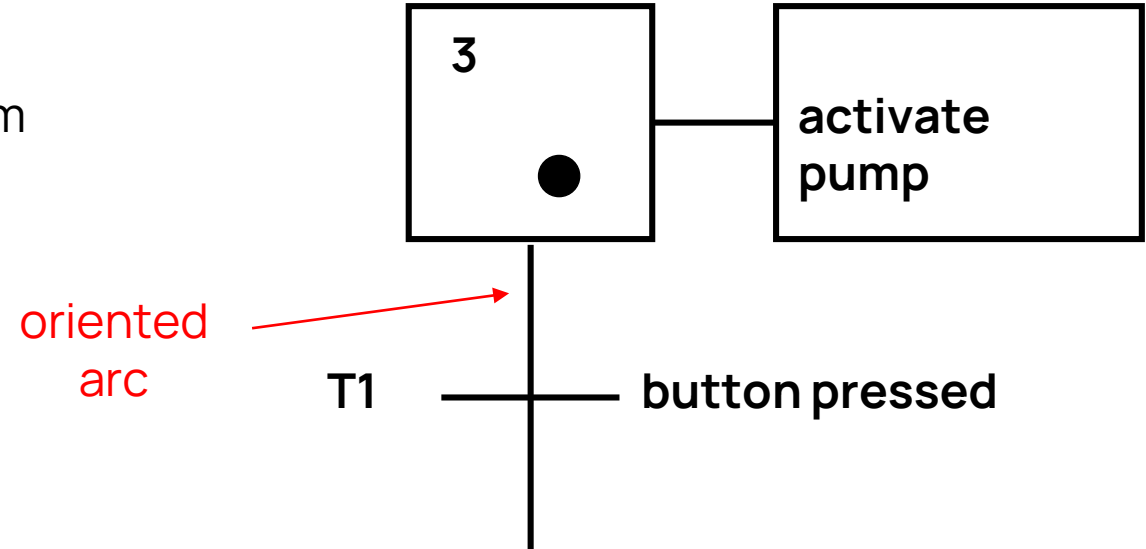
A transition is said to be **enabled** if all the upstream steps are active.

A transition is said to be **superable** if it is enabled and the condition associated with it takes on the “true” value (in this case it is also said that the transition can trigger).

The oriented arc

An **oriented arc** in a SFC:

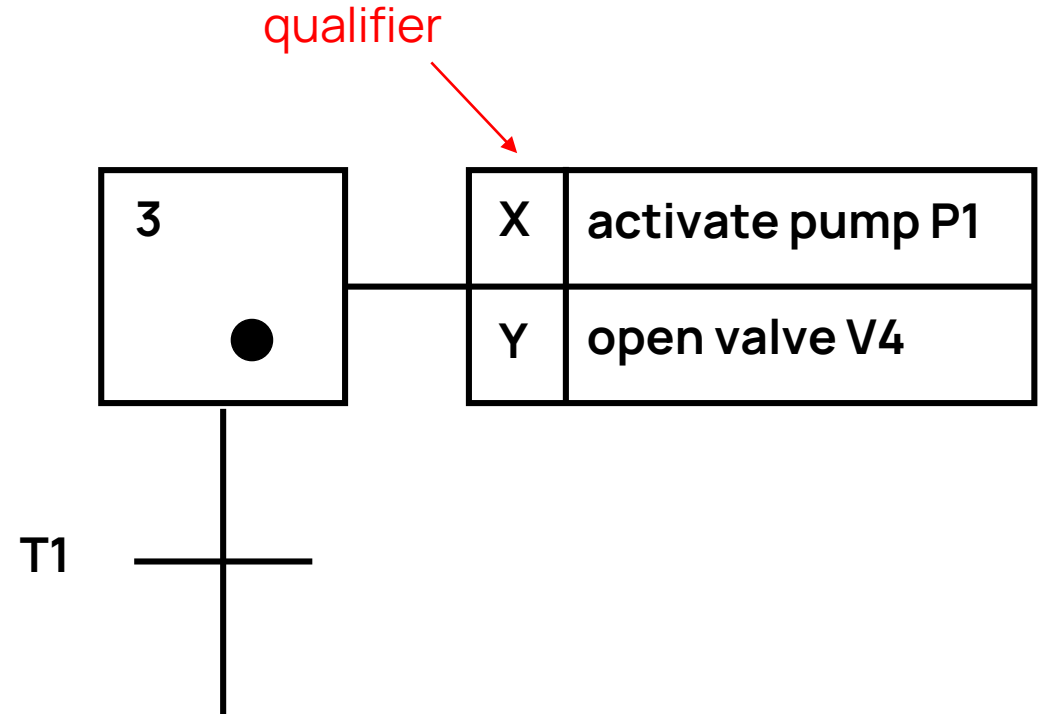
- is represented by **segments** connected with 90 deg angles
 - links **steps to transitions** or vice versa
 - the directional arrow can be omitted if it is from top to bottom.
-
- An oriented arc can never connect a step to another step
 - An oriented arc can never connect a transition to a transition



The action

Actions in a SFC are characterized by:

- The actions are associated with the steps and are the means with which the SFC scheme acts, on the variables inside the PLC and/or interacting with the outside.
- A step can have **multiple actions** associated with it, in which case they are performed in parallel.
- Actions are equipped with attributes (**qualifiers**) to describe unambiguously the temporal trend of the effect they produce.



Qualifiers of an action

Actions associated to steps can be of several types depending on the expected temporal behavior. To clarify this aspect, **qualifiers** are associated to each action:

N (normal or not-stored)

A continuous action, executed as long as the step is active. It is the default, if the qualifier is not specified

L (time-limited)

A time-limited continuous action executed for a limited (specified) amount of time or for the time when the step is active (if less than the specified time duration)

D (delayed)

The action is delayed in time, i.e. it is executed after the amount of time indicated, if the step is active

Qualifiers of an action

P (pulse)

An action to be executed just once, when the step is active.

S (stored or set)

The action begins when the step is activated and lasts, regardless of how long the step remains active, until it is reset with an equal action but with a qualifier **R**

R (reset)

Terminates a stored action

DS (delayed and stored)

The action begins after an assigned time interval since the step has become active, if the step is still active at that time. If it begins, the action lasts until it is reset with an equal but **R**-type action.

SL (stored and time-limited)

The action begins when the step becomes active and ends when it is reset with an equal but **R**-type action or when an assigned amount of time has elapsed.

Rules of evolution of a SFC

As already said:

- A transition is said to be **enabled** if all the upstream steps are active.
- A transition is said to be **superable** if it is enabled and the condition associated with it takes on the “true” value (in this case it is also said that the transition can trigger).

There are a couple of important rules to keep in mind:

- **Rule 1** When a transition is superable, it is passed: all upstream steps are deactivated at once and all downstream steps are activated at once.
- **Rule 2** If several transitions are superable at a certain time, they are all passed at the same time

Programming the SFC

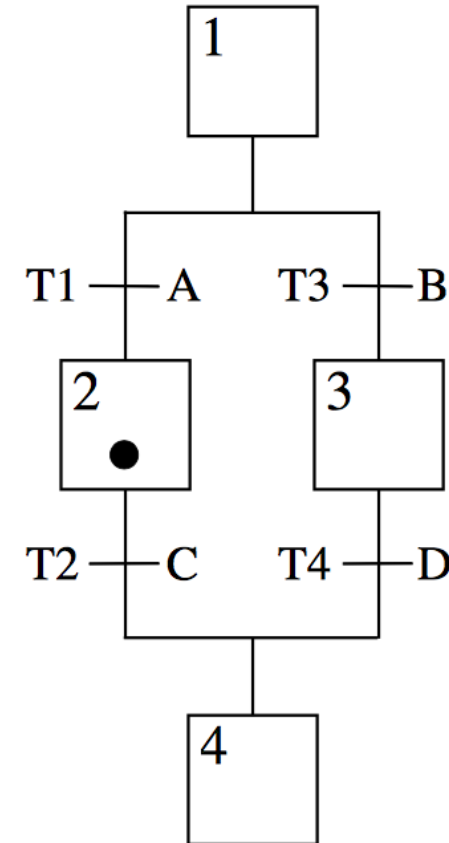
02

SFC: typical programming structures

if-then-else

Two branch nodes are used:

- a **node of choice**, i.e. a step followed by multiple transitions with mutually exclusive conditions
- a **convergence node**, i.e. multiple transitions that lead in (that is, by triggering they activate the) same step



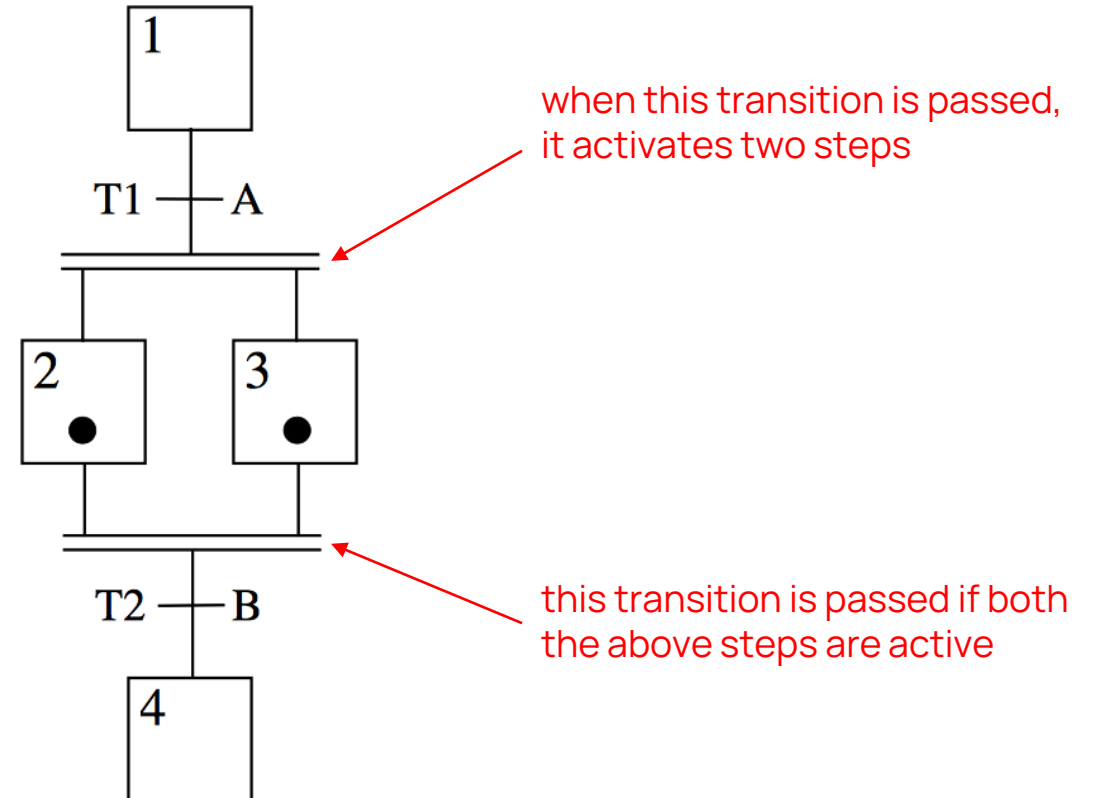
A and B are **mutually exclusive conditions**

SFC: typical programming structures

parallelism and rendezvous

Two branch nodes are used, indicated in this case with **double bars**:

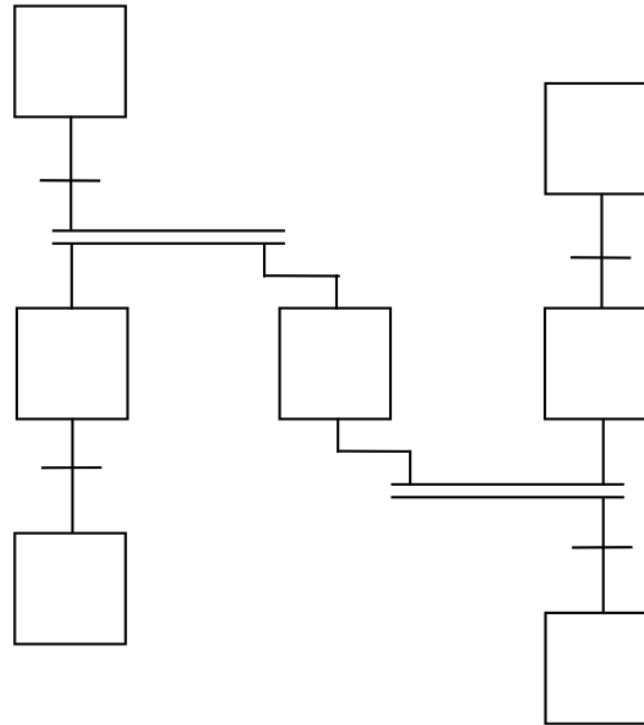
- a node of **parallelism**, that is a transition followed by multiple steps (which are all activated when the transition is passed)
- a **concurrency (rendezvous)** node, that is, a transition preceded by several steps (and therefore superable only if they are all active).



SFC: typical programming structures

local synchronization

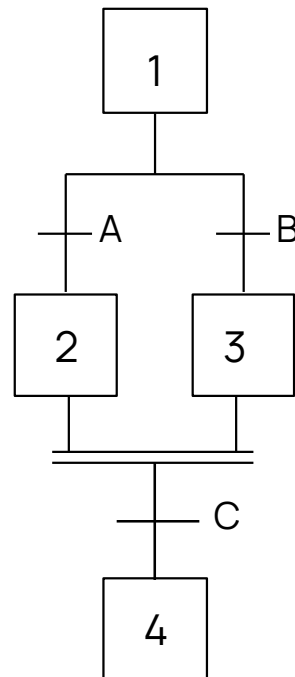
It is a structure in which two sequences have to wait each other at a given step



The sequence to the right has to wait for the sequence to the left

SFC: typical programming errors

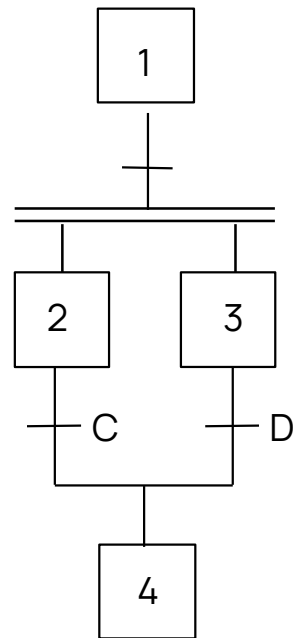
choice among sequences followed by synchronization of the sequences



- A and B are mutually exclusive conditions
- as a consequence transition C will never be superable

SFC: typical programming errors

Parallelism of sequences followed by convergence of the sequences



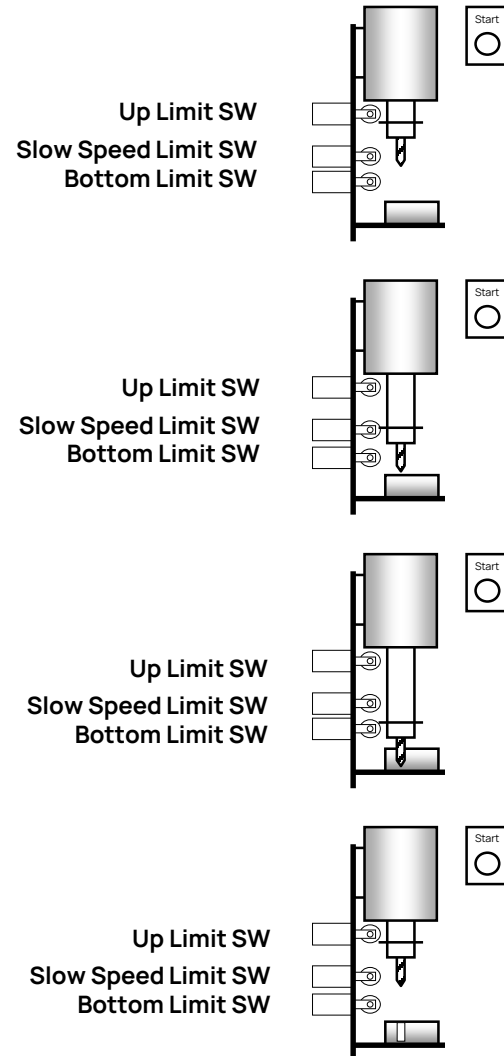
Step 4 might be activated twice



Examples

03

Sequential function chart: example 1

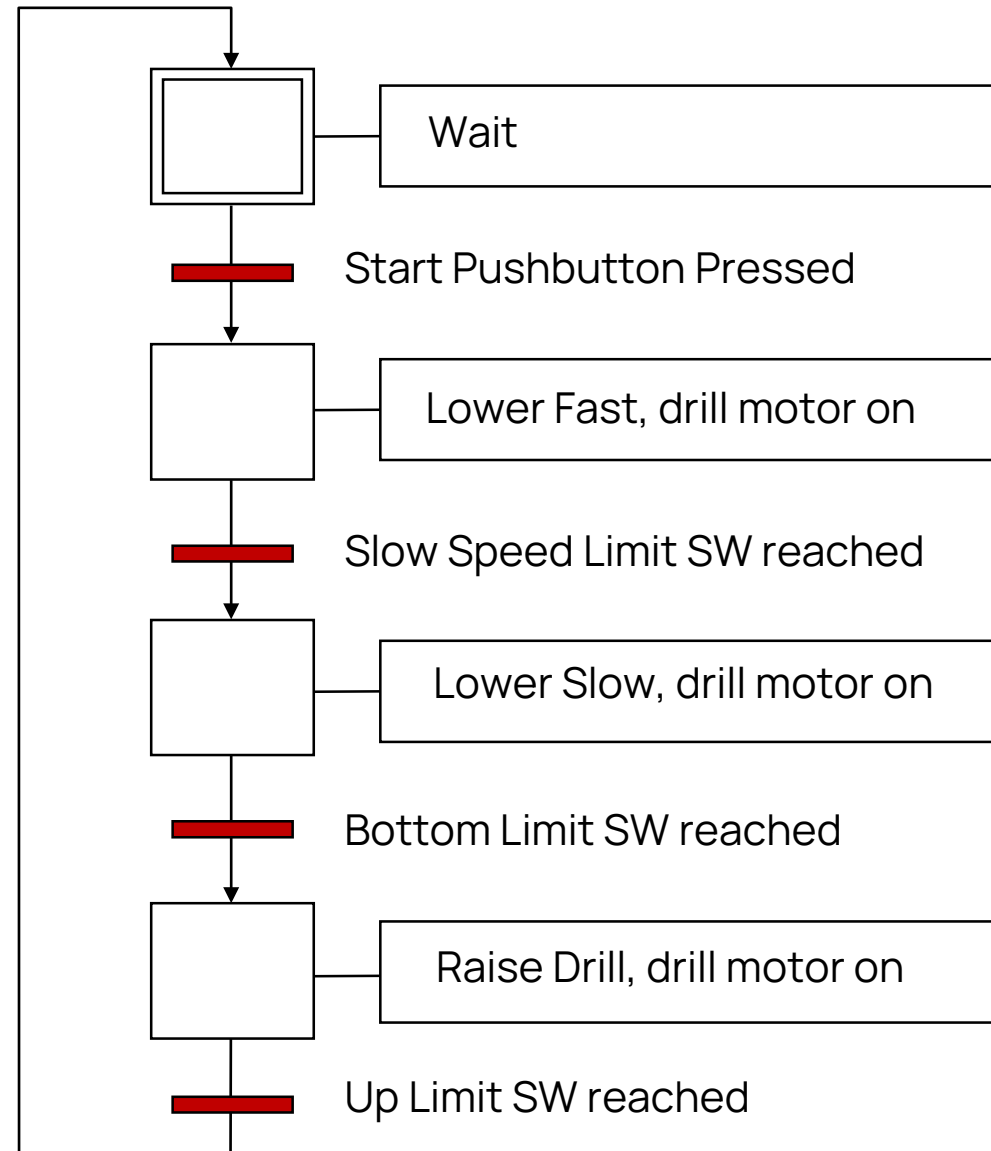


Consider the **automatic drill** depicted in the picture:

When the operator presses a start button, the drill starts approaching the piece at high speed, until it reaches a switch where the speed has to be lowered.

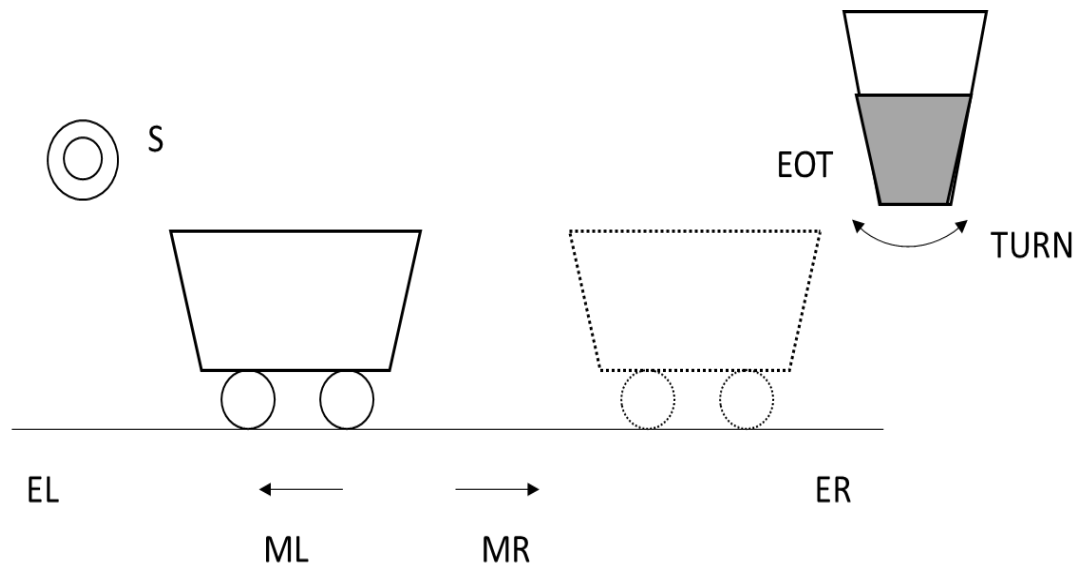
The drill then moves all the way to the bottom, until a switch is reached that corresponds to the bottom limit. The drill then moves upright, until it reaches the upper limit switch.

Sequential function chart: example 1



Sequential function chart: example 2

Consider the **automatic cart** depicted in the picture:

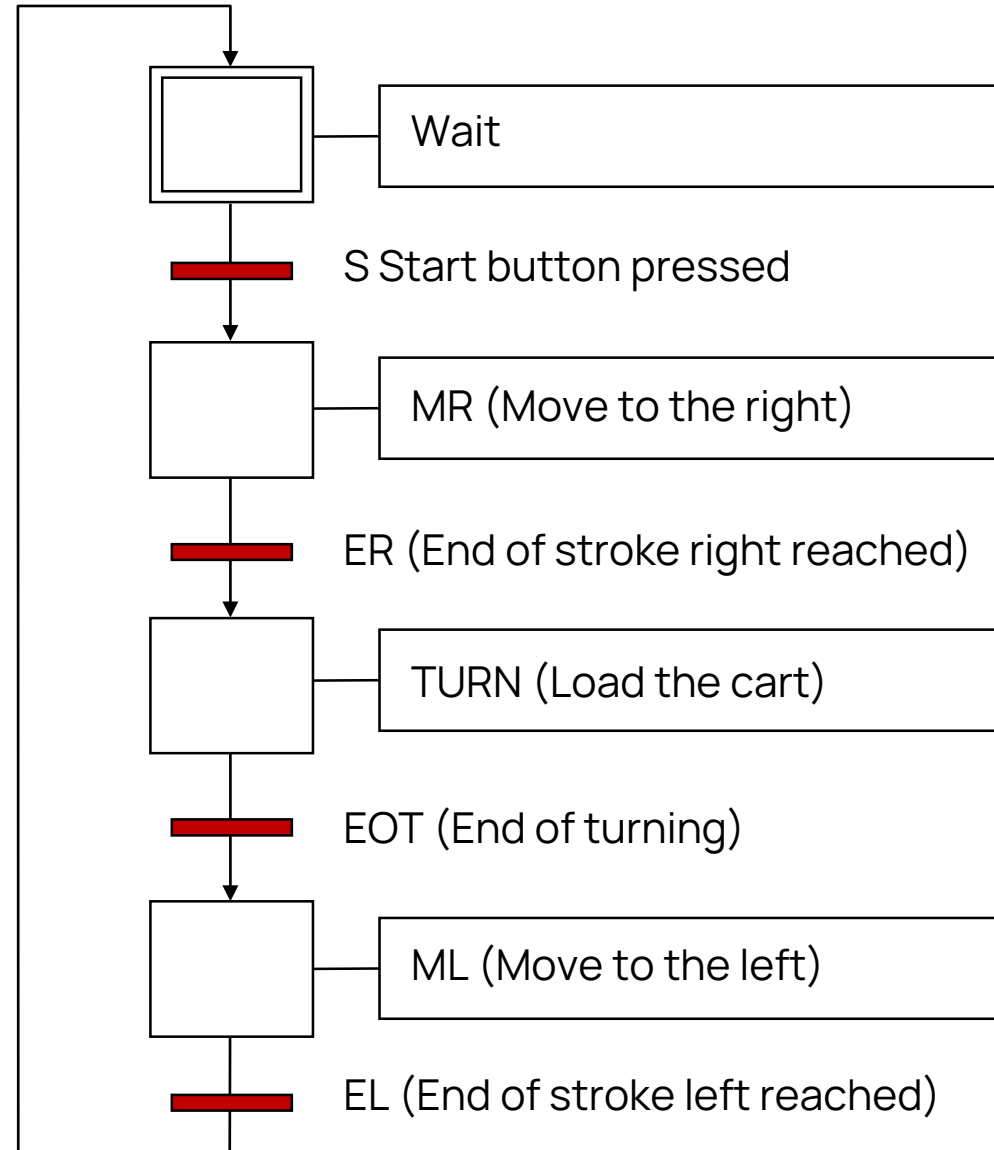


When the operator presses a button, the cart has to move to the right until it reaches the end of a rail, then it is filled with some material by turning a rotating tank.

The cart has then to move all the way to the left to be unloaded. Three end of stroke signals (two for the carts: **ER**, **EL**, one for the tank: **EOT**) and one signal corresponding to the button pressed (**S**) are available.

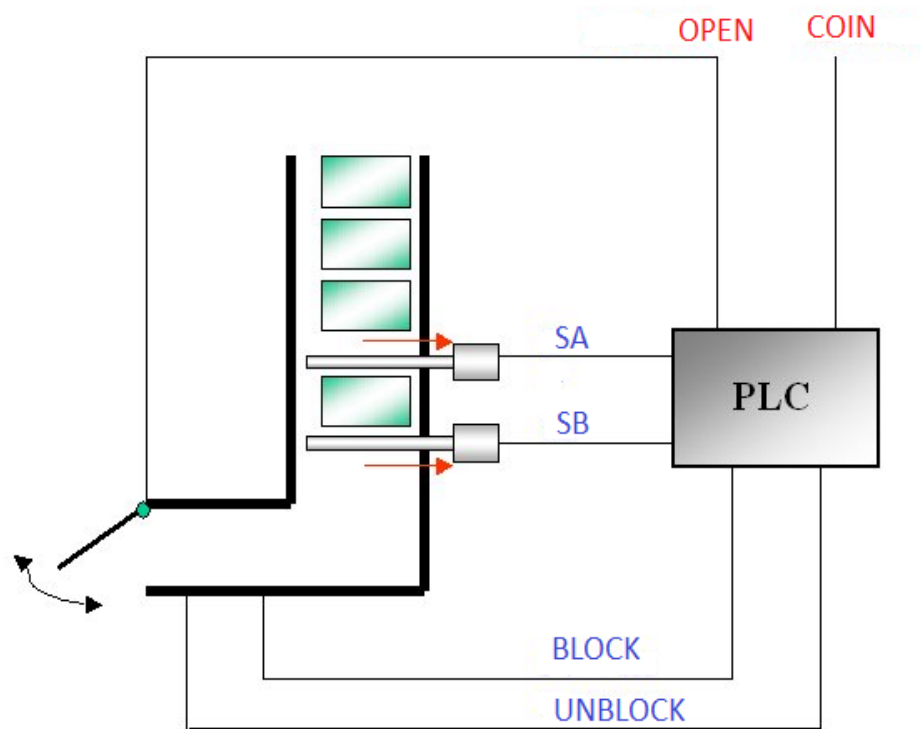
Two commands for moving the cart to the right (**MR**) and to the left (**ML**) and one for turning the tank (**TURN**) are available

Sequential function chart: example 2



Sequential function chart: example 3

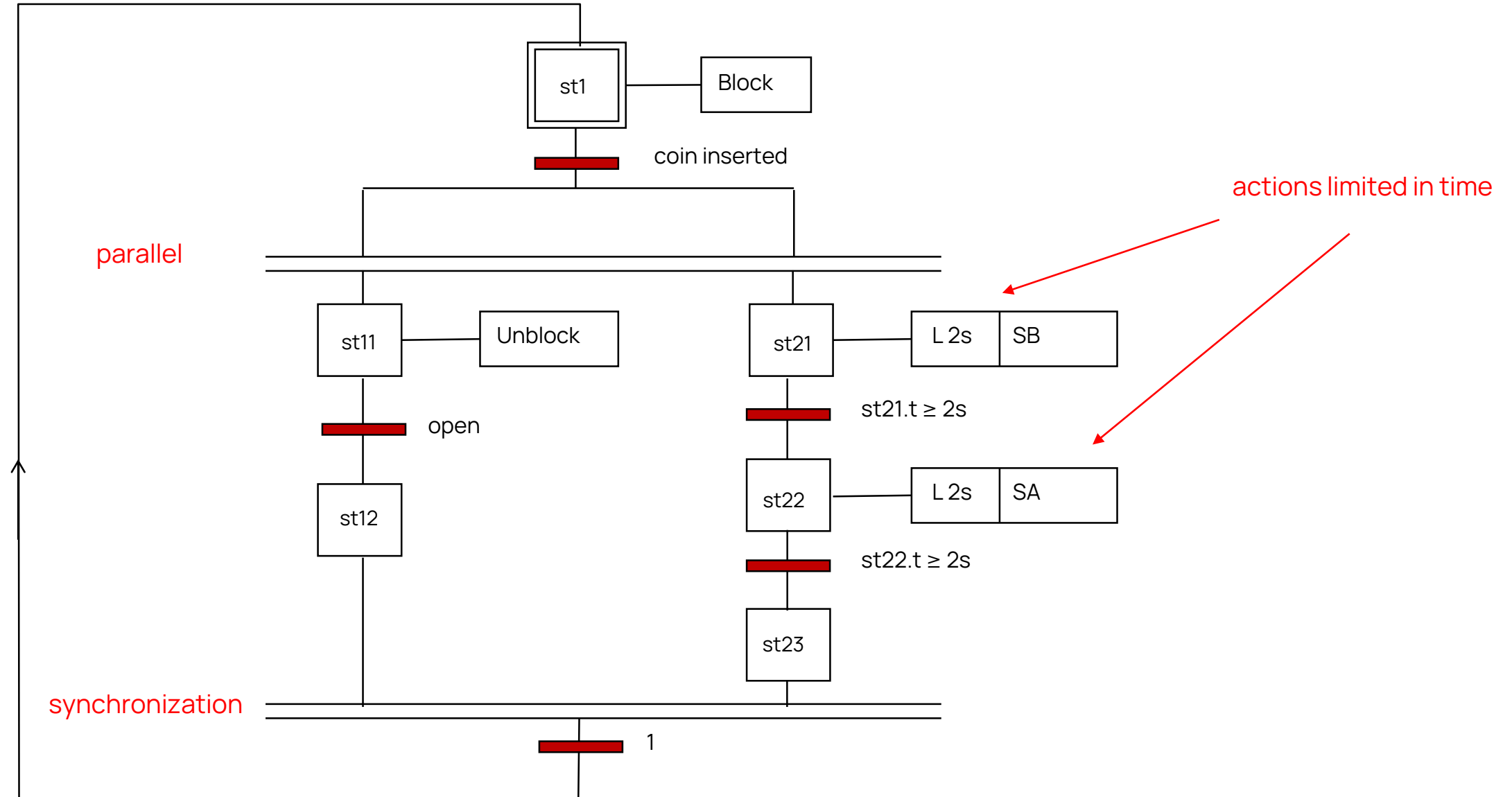
Consider the **vending machine** in the picture:



When the user inserts a **coin**, the piston B is moved to the right through the command **SB** in order to release a can and the same for the piston A (command **SA**) in order to make a can available for the next user. Only one can is released for each coin inserted.

The door of the output compartment is unblocked (command **UNBLOCK**), the user opens the door to pick the can, and then the door is blocked again (command **BLOCK**) after the user closes it

Sequential function chart: example 3





POLITECNICO
MILANO 1863

DIPARTIMENTO DI ELETTRONICA
INFORMAZIONE E BIOINGEGNERIA

Contatti

Paolo Rocco
paolo.rocco@polimi.it